

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC ỨNG DỤNG
BỘ MÔN TOÁN ỨNG DỤNG



BÀI TẬP LỚN
ĐẠI SỐ TUYẾN TÍNH (MT1007)

PHÂN TÍCH PCA ĐỂ NHẬN DIỆN
KHUÔN MẶT

Nhóm 3 - Lớp DT01 - Học kỳ 233

Giảng viên hướng dẫn: ThS. Nguyễn Hữu Hiệp

STT	Họ và tên	MSSV	Ghi chú
1	Huỳnh Nhật Huy	2348015	
2	Huỳnh Phát Lộc	2211917	
3	Huỳnh Phước An	2310010	
4	Lâm Chí Vĩ	2035075	
5	Lâm Hoàng Tân	2213054	Nhóm trưởng
6	Lâm Thành Đạt	2153280	
7	Lê Hồ Thiên Kim	2348072	

Thành phố Hồ Chí Minh, tháng 7 năm 2024

Danh sách thành viên

STT	Họ và tên	MSSV	Nhiệm vụ	Đóng góp
1	Hình Nhật Huy	2348015	-	0%
2	Huỳnh Phát Lộc	2211917	Lời mở đầu, chương 2	100%
3	Huỳnh Phước An	2310010	Chương 1, chương 3	100%
4	Lâm Chí Vĩ	2035075	-	0%
5	Lâm Hoàng Tân	2213054	Chương 4, tổng hợp, làm báo cáo và slide	100%
6	Lâm Thành Đạt	2153280	Chương 1, chương 3	100%
7	Lê Hồ Thiên Kim	2348072	Tổng kết, chương 2	100%

Mục lục

Lời mở đầu	1
1 Cơ sở lý thuyết	2
1.1 Kỳ vọng	2
1.2 Độ lệch chuẩn	2
1.3 Phương sai	2
1.4 Hiệp phương sai	3
1.5 Ma trận hiệp phương sai	3
1.6 Trị riêng và vector riêng	4
1.7 Phân tích thành phần chính	5
2 Thuật toán PCA	7
2.1 Tổng quan thuật toán	7
2.2 Triển khai thuật toán	8
2.3 Áp dụng thuật toán vào nhận diện khuôn mặt	9
2.4 Ưu điểm và nhược điểm của thuật toán	9
2.5 Ứng dụng	10
3 Nhận diện sinh viên vào Trường Đại học Bách khoa	11
3.1 Sơ đồ khối	11
3.2 Quy trình	11
3.3 Nhận xét	12
4 Chương trình nhận diện khuôn mặt bằng Python	13
4.1 Ý tưởng	13
4.2 Triển khai	13
4.2.1 Tiền xử lý dữ liệu	13
4.2.2 Huấn luyện mô hình PCA	13
4.2.3 Nhận diện khuôn mặt	14
4.2.4 Đánh giá hiệu suất	14
4.3 Nguồn dữ liệu và nguồn code	14
4.4 Giải thích các câu lệnh	20
4.5 Kiểm tra và phân tích kết quả	20
Tổng kết	27
Tài liệu tham khảo	28

Lời mở đầu

Với sự phát triển của xã hội, khoa học công nghệ trên thế giới đã và đang phát triển một cách rõ rệt, các ứng dụng trí tuệ nhân tạo cũng ngày càng trở nên phổ biến và được ứng dụng trong nhiều lĩnh vực khác nhau. Song song đó, các vấn đề liên quan đến an ninh bảo mật cũng được yêu cầu khắt khe hơn trên thế giới.

Hiện nay, các bài toán nhận dạng con người rất được quan tâm, cụ thể là bài toán nhận diện khuôn mặt, bởi vì nhận diện khuôn mặt là cách mà con người có thể phân biệt với nhau. Không chỉ vậy, việc nhận diện khuôn mặt qua hình ảnh dựa trên việc thu thập, xử lý thông tin cũng đang được quan tâm và ứng dụng rộng rãi trên toàn cầu. Kết hợp với sự phát triển của khoa học công nghệ, các hệ thống nhận diện khuôn mặt đã ra đời nhằm tạo điều kiện thuận lợi cho các bài toán nhận diện khuôn mặt từ ảnh số.

Trong lĩnh vực này, một trong những phương pháp được sử dụng phổ biến là phương pháp Phân tích thành phần chính (PCA - Principal Component Analysis). Phương pháp PCA là một phương pháp phân tích, trích xuất thông tin quan trọng từ dữ liệu đầu vào, giúp giảm chiều dữ liệu mà vẫn giữ được những thông tin quan trọng.

Chính vì vậy, nhóm quyết định thực hiện đề tài ***"Phân tích PCA để nhận diện khuôn mặt"*** nhằm phân tích cũng như áp dụng thuật toán PCA vào quy trình nhận diện khuôn mặt và xây dựng chương trình trên phần mềm Python.

1 Cơ sở lý thuyết

1.1 Kỳ vọng

Giá trị kỳ vọng (expected value), hoặc giá trị mong đợi, hoặc kỳ vọng toán học, hoặc trung bình (mean) của một biến ngẫu nhiên là trung bình có trọng số là xác suất của tất cả các giá trị cụ thể của biến đó.

Công thức tính kỳ vọng

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i \quad (1)$$

Trong đó:

- $\bar{x}$ là giá trị trung bình (kỳ vọng) của các quan sát
- x_i là kết quả quan sát lần thứ i của mẫu
- n là số lượng quan sát trong mẫu

1.2 Độ lệch chuẩn

Độ lệch chuẩn (Standard Deviation) là một đại lượng thống kê mô tả dùng để đo mức độ phân tán của một tập dữ liệu đã được lập thành bảng tần số.

Công thức tính độ lệch chuẩn

$$SD = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2} \quad (2)$$

Trong đó:

- SD là độ lệch chuẩn
- x_i là kết quả quan sát lần thứ i của mẫu
- $\bar{x}$ là giá trị trung bình của các quan sát
- n là số lượng quan sát trong mẫu

1.3 Phương sai

Phương sai (Variance) là một thước đo sự phân tán thống kê của biến đó, được tính bằng giá trị trung bình của các bình phương của chênh lệch giữa giá trị cụ thể và giá trị trung

binh. Phương sai càng nhỏ thì các điểm dữ liệu càng gần kỳ vọng. Ngược lại, khi phương sai càng lớn thì dữ liệu càng phân tán.

Công thức tính phương sai

$$\sigma^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2 \quad (3)$$

Trong đó:

- x_i là kết quả quan sát lần thứ i của mẫu
- $\bar{x}$ là giá trị trung bình của các quan sát
- n là số lượng quan sát trong mẫu

1.4 Hiệp phương sai

Hiệp phương sai (Covariance) là độ đo sự biến thiên cùng nhau của hai biến ngẫu nhiên. Nếu 2 biến có xu hướng thay đổi cùng nhau, tức là khi một biến có giá trị cao hơn giá trị kỳ vọng thì biến kia có xu hướng cũng cao hơn giá trị kỳ vọng, thì hiệp phương sai giữa hai biến này có giá trị dương. Mặt khác, nếu một biến nằm trên giá trị kỳ vọng còn biến kia có xu hướng nằm dưới giá trị kỳ vọng, thì hiệp phương sai của hai biến này có giá trị âm. Nếu hai biến này độc lập với nhau thì giá trị bằng 0.

Công thức tính hiệp phương sai

$$Cov(x, y) = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{n-1} \quad (4)$$

Trong đó:

- x_i là kết quả quan sát lần thứ i của mẫu
- y_i là giá trị y tương ứng với x_i trong tập dữ liệu
- $\bar{x}, \bar{y}$ lần lượt là giá trị trung bình tương ứng với x và y
- n là số lượng quan sát trong mẫu

1.5 Ma trận hiệp phương sai

Ma trận hiệp phương sai của tập hợp biến ngẫu nhiên là một ma trận vuông, trong đó các phần tử nằm trên đường chéo lần lượt là phương sai của các biến. Còn các phần tử còn lại, tức là những phần tử không nằm trên đường chéo, là hiệp phương sai của đôi 2 biến

ngẫu nhiên trong tập hợp.

Mọi phần tử trên đường chéo của ma trận hiệp phương sai là các số không âm. Chúng cũng chính là phương sai của từng chiều của dữ liệu.

Các phần tử ngoài đường chéo có thể dương, hoặc âm, hoặc bằng 0. Khi nó bằng 0, tức là hiệp phương sai bằng 0, ta nói rằng hai thành phần tương ứng trong dữ liệu không tương quan với nhau.

Công thức ma trận hiệp phương sai

$$\Sigma = \begin{pmatrix} \text{var}(x_1) & \text{Cov}(x_1, x_2) & \dots & \text{Cov}(x_1, x_{n-1}) & \text{Cov}(x_1, x_n) \\ \text{Cov}(x_2, x_1) & \text{var}(x_2) & \dots & \text{Cov}(x_2, x_{n-1}) & \text{Cov}(x_2, x_n) \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ \text{Cov}(x_{n-1}, x_1) & \text{Cov}(x_{n-1}, x_2) & \dots & \text{var}(x_{n-1}) & \text{Cov}(x_{n-1}, x_n) \\ \text{Cov}(x_n, x_1) & \text{Cov}(x_n, x_2) & \dots & \text{Cov}(x_n, x_{n-1}) & \text{var}(x_n) \end{pmatrix} \quad (5)$$

Trong đó:

- $\text{Cov}(x_1, y_1)$ là hiệp phương sai của hai biến x_1 và y_1
- $\text{var}(x)$ là phương sai của x

1.6 Trị riêng và vector riêng

Khi một ma trận vuông A cấp n bất kỳ nhân với vector x (vector x khác vector 0), ta thu được kết quả là một bội số vô hướng của vector x . Hay nói cách khác

$$Ax = \lambda x \quad (6)$$

Khi đó, số vô hướng này được gọi là trị riêng λ của vector x và vector x được gọi là vector riêng tương ứng với trị riêng λ .

Phương trình đặc trưng

Cho một ma trận vuông $A \in M_n(\mathbb{K})$. Khi đó, phương trình đặc trưng của A được định nghĩa là

$$\det(A - \lambda I) = 0 \quad (7)$$

Tính chất của trị riêng và vector riêng

- Một trị riêng có thể có nhiều vector riêng
- Mỗi vector riêng chỉ ứng với một trị riêng duy nhất
- Mọi ma trận vuông bậc n đều có n trị riêng (kể cả lặp) và có thể là các số phức.
- Nếu x là vector riêng của A ứng với λ , thì kx ($k \neq 0$) cũng là vector riêng ứng với trị riêng đó
- Tổng tất cả các trị riêng của A bằng với vết của ma trận A , tức là bằng với tổng các phần tử trên đường chéo của A
- Tích tất cả các trị riêng của A bằng với $\det(A)$
- Tổng tất cả các bội đại số của các trị riêng bằng với cấp của A
- Tổng tất cả các bội hình học của các trị riêng bằng với số vector độc lập tuyến tính cực đại
- Nếu λ là trị riêng của ma trận A , thì λ^m là trị riêng của ma trận A^m , với $m \in \mathbb{N}$

Phương pháp tìm trị riêng và vector riêng

Bước 1: Tìm trị riêng

- Lập phương trình đặc trưng $\det(A - \lambda I) = 0$
- Tính định thức, giải phương trình
- Tất cả nghiệm của phương trình đặc trưng vừa tìm được là tất cả trị riêng của ma trận A

Bước 2: Tìm vector riêng

- Giải hệ phương trình $(A - \lambda_1 I)x = 0$ tương ứng với trị riêng λ_1
- Tất cả các nghiệm khác 0 của hệ là tất cả các vector riêng x của ma trận A ứng với trị riêng λ_1
- Tương tự tìm vector riêng của ma trận A ứng với các trị riêng còn lại

1.7 Phân tích thành phần chính

Phép phân tích thành phần chính (Principal Components Analysis - PCA) là một thuật toán thống kê sử dụng phép biến đổi trực giao để biến đổi một tập hợp dữ liệu từ một không gian nhiều chiều sang một không gian mới ít chiều hơn (2 hoặc 3 chiều) nhằm tối ưu hóa việc thể hiện sự biến thiên của dữ liệu.

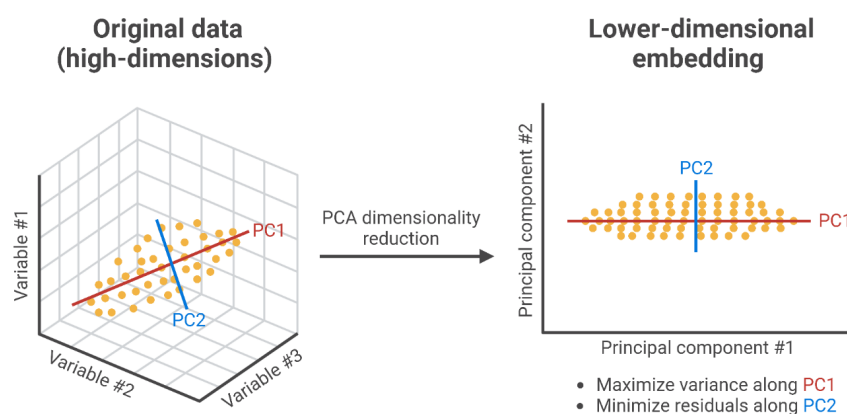
Đặc điểm của phép phân tích thành phần chính

- Giảm số chiều của không gian chứa dữ liệu khi nó có số chiều lớn, không thể thể hiện trong không gian 2 hay 3 chiều
- Xây dựng những trục tọa độ mới, thay vì giữ lại các trục của không gian cũ, nhưng lại có khả năng biểu diễn dữ liệu tốt tương đương, và đảm bảo độ biến thiên của dữ liệu trên mỗi chiều mới
- Tạo điều kiện để các liên kết tiềm ẩn của dữ liệu có thể được khám phá trong không gian mới, mà nếu đặt trong không gian cũ thì khó phát hiện vì những liên kết này không thể hiện rõ
- Đảm bảo các trục tọa độ trong không gian mới luôn trực giao đôi một với nhau, mặc dù trong không gian ban đầu các trục có thể không trực giao

2 Thuật toán PCA

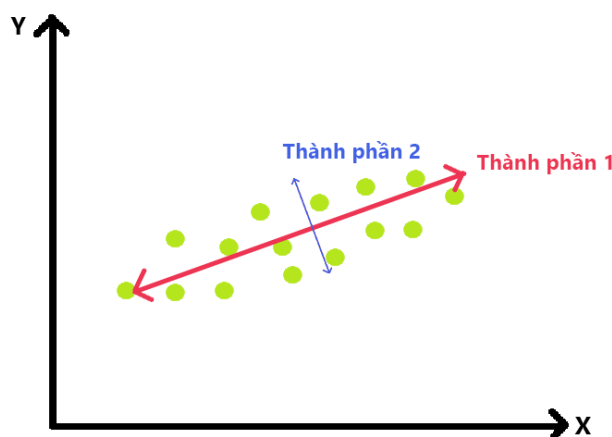
2.1 Tổng quan thuật toán

Principal Component Analysis (PCA), hay phân tích thành phần chính, là một kỹ thuật phân tích dữ liệu phổ biến trong học máy và thống kê, đặc biệt khi đối mặt với những bộ dữ liệu lớn. PCA được sử dụng để giảm thiểu số chiều của dữ liệu mà vẫn không mất đi thông tin cần thiết.



Hình 1: Tổng quan thuật toán PCA

PCA sử dụng các phép biến đổi trực giao để biến đổi một tập hợp dữ liệu từ một không gian nhiều chiều sang một không gian có ít chiều hơn (2 đến 3 chiều) nhằm tối ưu hóa việc thể hiện sự biến thiên của dữ liệu. Các chiều trong ma trận đầu ra còn được gọi là **thành phần chính**. Các thành phần chính ở đây thực chất là các vector độc lập tuyến tính được chọn sao cho khi chiếu các điểm dữ liệu lên vector đó thì các điểm dữ liệu có sự biến động lớn nhất (phương sai lớn nhất).



Hình 2: Minh họa cho việc chọn thành phần chính

Từ Hình 2, giả sử ta chọn 2 thành phần chính. Ta có vector màu đỏ có mức độ biến thiên lớn nhất, nên ta chọn vector đó làm thành phần 1, sau đó đến vector màu xanh có mức độ biến thiên thấp hơn làm thành phần 2. Việc ưu tiên chọn thành phần có mức độ dữ liệu biến thiên lớn nhất sẽ giúp cho việc phân loại dữ liệu dễ dàng hơn.

Với X là ma trận các vector biểu diễn dữ liệu gốc, Y là ma trận các vector biểu diễn lại dữ liệu. Khi đó, ta gọi P là phép biến đổi tuyến tính khi áp dụng thuật toán PCA sao cho

$$P \times X = Y \quad (8)$$

2.2 Triển khai thuật toán

1. Tính toán giá trị trung bình bằng cách tính trung bình các giá trị đầu vào

$$\bar{X} = \frac{1}{N} \sum_{i=1}^N X_i \quad (9)$$

2. Chuẩn hoá dữ liệu để giải quyết các trường hợp biến của dữ liệu có đơn vị đo lường khác nhau. Việc chuẩn hoá là bước quan trọng và ảnh hưởng trực tiếp tới các bước kế tiếp. Ta chuẩn hoá bằng cách trừ đi giá trị trung bình

$$\tilde{X} = X_i - \bar{X}, i = 1, 2, \dots, N \quad (10)$$

3. Tính toán ma trận hiệp phương sai

$$\Sigma = \frac{1}{n-1} (\tilde{X} - \bar{X})^T (\tilde{X} - \bar{X}) \quad (11)$$

4. Từ ma trận hiệp phương sai thu được, ta tìm trị riêng dựa trên phương trình đặc trưng

$$\det(\Sigma - \lambda I) = 0 \quad (12)$$

Sau đó, ta giải hệ phương trình với trị riêng tìm được để tìm vector riêng x tương ứng

$$(\Sigma - \lambda I)x = 0 \quad (13)$$

5. Sau khi tính toán các giá trị riêng và vector riêng của ma trận hiệp phương sai, cần sắp xếp các thành phần chính dựa trên giá trị riêng và chọn ra các thành phần chính

quan trọng nhất. Giá trị riêng càng lớn thì vector riêng (thành phần chính) tương ứng càng giải thích nhiều cho sự biến thiên của dữ liệu. Nói cách khác, ta chọn cơ sở E được sinh ra bởi n vector riêng v_i ứng với n giá trị riêng λ_i lớn nhất.

6. Tiến hành chiếu vector $\tilde{X}$ lên cơ sở E thu được. Sau đó tìm trọng số của $\tilde{X}_i$ theo các vector riêng được chọn ở cơ sở E và phân tích $\tilde{X}_i$ thành tổ hợp tuyến tính giữa các tích trọng số thứ i với vector riêng v_i

$$\tilde{X}_i = \sum_{i=1}^k \omega_i v_i \quad (14)$$

Sau khi thực hiện, ta thu được các vector trọng số $\Omega_i = [(\omega_i)_1, (\omega_i)_2, \dots, (\omega_i)_k]$

2.3 Áp dụng thuật toán vào nhận diện khuôn mặt

1. Tách phần mặt từ các ảnh của cơ sở dữ liệu và chuẩn hóa lại kích cỡ của các ảnh cho đồng đều.
2. Áp dụng thuật toán PCA được triển khai ở Phần 2.2 để chuẩn bị cho việc nhận diện.
3. Để nhận dạng, ta tính trọng số của ảnh đầu vào và tính độ lệch thông qua khoảng cách Euclidean

$$D_i = ||\Omega - \Omega_i|| \quad (15)$$

Sau đó, ta chọn ra giá trị D_i nhỏ nhất và trả về khuôn mặt tương ứng với giá trị i .

2.4 Ưu điểm và nhược điểm của thuật toán

Về **ưu điểm**, thuật toán PCA giúp giảm chiều dữ liệu, nhằm làm giảm độ phức tạp khi áp dụng dữ liệu vào mô hình và thời gian tính toán, đặc biệt khi xử lý dữ liệu lớn. Bên cạnh đó, PCA còn trực quan hoá dữ liệu và loại bỏ các thành phần chứa nhiễu và không đáng kể, đồng thời cải thiện hiệu suất của các thuật toán học máy. Quan trọng nhất, PCA tạo ra các thành phần chính là các tổ hợp tuyến tính của các biến gốc nhưng không tương quan với nhau, giúp đơn giản hóa phân tích và giải thích mô hình.

Bên cạnh đó, về **nhược điểm**, khi làm giảm chiều dữ liệu có thể dẫn đến mất mát một số thông tin, đặc biệt nếu các thành phần chính nhỏ nhưng vẫn mang thông tin quan trọng bị bỏ qua. Ngoài ra, PCA yêu cầu dữ liệu đầu vào phải có cấu trúc và không có nhiều giá trị thiếu. Điều này đòi hỏi bước tiền xử lý dữ liệu phải được thực hiện cẩn thận.

2.5 Ứng dụng

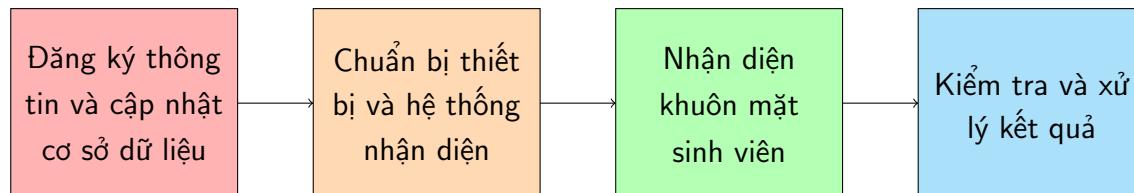
Trong lĩnh vực xử lý hình ảnh và thị giác máy tính, PCA được sử dụng để giảm số lượng pixel cần thiết để biểu diễn một hình ảnh, giảm kích thước tệp tin mà vẫn giữ được chất lượng hình ảnh.

Trong lĩnh vực công nghệ sinh học, PCA được dùng để phân tích biểu hiện gen, protein và dữ liệu sinh học khác và có thể giúp phân loại các bệnh dựa trên dữ liệu y tế phức tạp.

Trong lĩnh vực tài chính, PCA được sử dụng để giảm số lượng biến số trong các mô hình tài chính, giúp dự đoán rủi ro và lợi nhuận, phân tích khách hàng và nghiên cứu thị trường.

3 Nhận diện sinh viên vào Trường Đại học Bách khoa

3.1 Sơ đồ khối



Hình 3: Quy trình nhận diện tổng quát

3.2 Quy trình

1. Đăng ký thông tin và cập nhật cơ sở dữ liệu

- Đăng ký sinh viên mới: Khi sinh viên nhập học, họ sẽ đăng ký thông tin cá nhân như họ tên, mã số sinh viên, lớp học, ngày sinh, thông tin liên lạc và chụp ảnh khuôn mặt từ nhiều góc độ khác nhau để lưu trữ trong cơ sở dữ liệu. Các thông tin này sẽ được lưu trữ an toàn trong cơ sở dữ liệu của trường.
- Cập nhật thông tin: Thông tin và ảnh của sinh viên có thể được cập nhật định kỳ hoặc khi có sự thay đổi. Mỗi học kỳ hoặc năm học, sinh viên có thể cập nhật lại thông tin và chụp lại ảnh nếu cần. Ngoài ra, nếu sinh viên thay đổi ngoại hình, họ có thể yêu cầu cập nhật lại ảnh.

2. Chuẩn bị thiết bị và hệ thống nhận diện

- Camera giám sát: Camera được lắp đặt tại cổng vào trường, lối ra vào các khu vực quan trọng như thư viện, khu vực học tập. Đồng thời cần có độ phân giải cao, khả năng chụp ảnh trong điều kiện ánh sáng yếu, và tích hợp công nghệ nhận diện.
- Phần mềm nhận diện khuôn mặt: Phần mềm được cài đặt trên máy tính hoặc máy chủ, kết nối với các camera giám sát và cần được cấu hình để kết nối và truy xuất dữ liệu từ cơ sở dữ liệu khuôn mặt sinh viên.
- Cơ sở dữ liệu khuôn mặt: Dữ liệu khuôn mặt và thông tin cá nhân của sinh viên cần được lưu trữ trong môi trường bảo mật cao. Đồng thời, cơ sở dữ liệu cần có khả năng mở rộng để quản lý số lượng lớn sinh viên.

3. Nhận diện sinh viên

- Chụp ảnh: Khi sinh viên đến trường, camera sẽ tự động chụp lại hình ảnh của họ. Tuy nhiên, cần đảm bảo camera chụp ảnh khuôn mặt từ nhiều góc độ khác nhau để tăng độ chính xác.

- Phân tích khuôn mặt: Phần mềm sẽ phân tích hình ảnh mới chụp, sau đó sử dụng thuật toán PCA và so sánh với các hình ảnh trong cơ sở dữ liệu.
- Xác minh danh tính: Nếu phần mềm tìm thấy kết quả phù hợp, sinh viên sẽ được xác minh danh tính và cho phép vào trường. Ngược lại, hệ thống sẽ gửi cảnh báo tới nhân viên bảo vệ để kiểm tra thủ công nếu kết quả không phù hợp.

4. Kiểm tra và xử lý kết quả

- Kiểm tra kết quả: Kết quả nhận diện sẽ được hiển thị trên màn hình của nhân viên bảo vệ hoặc quản lý. Đồng thời, hệ thống sẽ tự động cảnh báo nếu phát hiện các trường hợp không khớp hoặc nghi ngờ.
- Xử lý ngoại lệ: Nhân viên bảo vệ sẽ kiểm tra giấy tờ tùy thân hoặc hỏi thêm thông tin để xác minh sinh viên hoặc ghi lại các trường hợp không nhận diện được hoặc có sự cố để phân tích và cải thiện hệ thống.

3.3 Nhận xét

Ưu điểm:

- Tự động hóa và tiết kiệm thời gian: Quy trình tự động giúp giảm thiểu thời gian kiểm tra thủ công, tăng tốc độ xác minh danh tính.
- Bảo mật cao: Hệ thống giúp kiểm soát an ninh tốt hơn, giảm nguy cơ người lạ vào trường.
- Theo dõi chính xác: Dữ liệu về lượt ra vào của sinh viên được ghi nhận chính xác, giúp quản lý dễ dàng hơn.

Nhược điểm:

- Chi phí đầu tư ban đầu cao: Mua sắm và lắp đặt camera, phần mềm, và cơ sở dữ liệu yêu cầu chi phí đầu tư lớn.
- Vấn đề về quyền riêng tư: Cần đảm bảo dữ liệu cá nhân của sinh viên được bảo mật và sử dụng đúng mục đích, tránh việc lạm dụng.
- Độ chính xác phụ thuộc vào nhiều yếu tố: Điều kiện ánh sáng, góc chụp, và thay đổi ngoại hình của sinh viên có thể ảnh hưởng đến độ chính xác của hệ thống.

4 Chương trình nhận diện khuôn mặt bằng Python

4.1 Ý tưởng

Để thực hiện chương trình nhận diện khuôn mặt bằng thuật toán PCA, ta cần tổng quát các bước để dễ dàng xây dựng chương trình:

1. Thu thập dữ liệu bao gồm ảnh khuôn mặt, xử lý ảnh và tách thành hai tập, bao gồm tập training và tập testing.
2. Ta tiến hành huấn luyện bằng thuật toán PCA, đồng thời biểu diễn ảnh bằng thành phần chính cả hai tập dữ liệu.
3. Đối với tập testing, ta tiến hành so sánh vector đặc trưng với tập training và đưa ra kết quả nhận diện.
4. Sau khi nhận diện, ta đánh giá hiệu quả bằng cách tính toán độ chính xác và phân tích kết quả thu được.

4.2 Triển khai

Để trực quan chương trình, nhóm quyết định chia ra thành 4 phần chính theo thứ tự là **Tiền xử lý dữ liệu**, **Huấn luyện mô hình PCA**, **Nhận diện khuôn mặt** và **Đánh giá hiệu suất**.

4.2.1 Tiền xử lý dữ liệu

Đây là giai đoạn đầu tiên của chương trình, với tập dữ liệu đầu vào được chia thành hai phần, tập training và tập testing. Trong đó, tỷ lệ tương ứng của tập training và tập testing lần lượt là 70% và 30%.

Sau đó, ta tiến hành kiểm tra dữ liệu của cả hai tập bằng cách chuyển đổi ảnh thành thang màu xám và chuẩn hoá kích thước.

Dữ liệu thu được sau khi kiểm tra sẽ áp dụng vào giai đoạn kế tiếp, là giai đoạn **Huấn luyện mô hình PCA**.

4.2.2 Huấn luyện mô hình PCA

Sau khi xử lý dữ liệu đầu vào, ta sử dụng tập training để tính toán cho mô hình. Việc tính toán được thực hiện lần lượt qua các bước như sau:

- Tính toán vector trung bình và chuẩn hoá dữ liệu.
- Tính toán ma trận hiệp phương sai.
- Tính toán vector riêng và trị riêng và sắp xếp theo thứ tự tăng dần.

- Lựa chọn **eigenfaces** dựa trên trị riêng và lưu trữ lại.

Cuối cùng, khi tính toán xong, ta cần lưu trữ lại **eigenfaces** được chọn và trị riêng tương ứng để sử dụng cho giai đoạn **Nhận diện khuôn mặt**.

4.2.3 Nhận diện khuôn mặt

Sau khi huấn luyện mô hình PCA cho tập training, ta sử dụng ảnh trong tập testing đã được xử lý ở giai đoạn **Tiền xử lý dữ liệu**.

Ta tiến hành chiếu ảnh khuôn mặt lên các **eigenfaces** đã được lưu trữ từ mô hình PCA đã huấn luyện.

Sau khi chiếu ảnh, ta tính toán khoảng cách Euclidean của vector chiếu của ảnh đầu vào và vector chiếu của từng ảnh, đồng thời so sánh với ngưỡng khoảng cách cho phép để tiến hành nhận diện.

4.2.4 Đánh giá hiệu suất

Sau khi nhận diện xong toàn bộ ảnh trong tập testing, ta tiến hành tính độ chính xác của thuật toán bằng cách lấy số ảnh dự đoán đúng chia tổng số ảnh trong tập testing.

Đồng thời, ta cũng có thể dự đoán được độ chính xác cao hơn bằng cách tìm ra ngưỡng khoảng cách phù hợp và tính toán độ chính xác.

4.3 Nguồn dữ liệu và nguồn code

Nguồn dữ liệu được truy cập từ <https://www.kaggle.com/datasets/kasikrit/att-database-of-faces>.

Nguồn code được thể hiện qua file `PCA.py` và được đăng tải trên [Google Drive](#). Trong đó, dữ liệu nhóm dùng để chạy chương trình được thể hiện qua folder `dataset`.

```
1 import os
2 import numpy as np
3 import matplotlib.pyplot as plot
4
5 dataset_path = 'C:/Users/HP/Downloads/database/'
6
7 img_width = 92
8 img_height = 112
9
10 #Training tensor
11 training_tensor = np.ndarray(shape = (98, img_height*img_width), dtype =
    ↳ np.float64)
```

```
12 for i in range (98):
13     img = plot.imread(dataset_path + 'training/' + str(i + 1) + '.pgm')
14     training_tensor[i,:] = np.array(img, dtype = 'float64').flatten()
15     plot.subplot(14, 7, i + 1)
16     plot.imshow(img, cmap = 'gray')
17     plot.tick_params(labelleft = False, labelbottom = False, bottom = False,
18         ↪ top = False, left = False, right = False, which = 'both')
19     plot.subplots_adjust(right = 0.6, top = 1)
20 plot.show()
21
22 #Testing tensor
23 testing_tensor = np.ndarray(shape = (42, img_height*img_width), dtype =
24     ↪ np.float64)
25 for i in range (42):
26     img = plot.imread(dataset_path + 'testing/' + str(i + 1) + '.pgm')
27     testing_tensor[i, :] = np.array(img, dtype = 'float64').flatten()
28     plot.subplot(14, 3, i + 1)
29     plot.imshow(img, cmap = 'gray')
30     plot.tick_params(labelleft = False, labelbottom = False, bottom = False,
31         ↪ top = False, left = False, right = False, which = 'both')
32     plot.subplots_adjust(right = 0.6, top = 1)
33 plot.show()
34 #PCA Algorithm
35 mean_face = np.zeros((1, img_height*img_width))
36 for i in training_tensor:
37     mean_face = np.add(mean_face, i)
38
39 mean_face = np.divide(mean_face, float(len(training_tensor))).flatten()
40 plot.imshow(mean_face.reshape(img_height, img_width), cmap = 'gray')
41 plot.tick_params(labelleft = False, labelbottom = False, bottom = False, top
42     ↪ = False, left = False, right = False, which = 'both')
43 plot.title("Mean face of training set")
44 plot.show()
45
46 normalize_training_tensor = np.ndarray(shape = (len(training_tensor),
47     ↪ img_height*img_width))
48 for i in range(len(training_tensor)):
```

```
47     normalize_training_tensor[i] = np.subtract(training_tensor[i], mean_face)
48
49 for i in range(len(training_tensor)):
50     img = normalize_training_tensor[i].reshape(img_height, img_width)
51     plot.subplot(14, 7, i + 1)
52     plot.imshow(img, cmap = 'gray')
53     plot.tick_params(labelleft = False, labelbottom = False, bottom = False,
54         ↪ top = False, left = False, right = False, which = 'both')
55     plot.subplots_adjust(right = 0.6, top = 1)
56 plot.show()
57
58 cov_matrix = np.dot(normalize_training_tensor, normalize_training_tensor.T)
59 eigenvalues, eigenvectors = np.linalg.eig(cov_matrix)
60
61 eig_pairs = [(eigenvalues[idx], eigenvectors[:, idx]) for idx in
62     ↪ range(len(eigenvalues))]
63 eig_pairs.sort(reverse = True)
64 eigvalues_sort = [eig_pairs[idx][0] for idx in range(len(eigenvalues))]
65 eigvectors_sort = [eig_pairs[idx][1] for idx in range(len(eigenvalues))]
66
67 var_comp_sum = np.cumsum(eigvalues_sort) / sum(eigvalues_sort)
68 num_comp = range(1, len(eigvalues_sort) + 1)
69
70 plot.title("Cumulative Proportion of Variance and Components Kept")
71 plot.xlabel("Principal Components")
72 plot.ylabel("Cumulative Proportion of Variance")
73 plot.scatter(num_comp, var_comp_sum)
74 plot.show()
75
76 print("Number of eigenvectors:", len(eigvectors_sort))
77 n_components = 80
78 print("The value of k:", n_components)
79
80 reduced_data = np.array(eigvectors_sort[:n_components]).transpose()
81
82 proj_data = np.dot(training_tensor.transpose(), reduced_data)
83 proj_data = proj_data.transpose()
84
85 for i in range(proj_data.shape[0]):
86     img = proj_data[i].reshape(img_height, img_width)
```

```
85     plot.subplot(10, 8, i + 1)
86     plot.imshow(img, cmap = 'gray')
87     plot.tick_params(labelleft = False, labelbottom = False, bottom = False,
88         ↪ top = False, left = False, right = False, which = 'both')
89     plot.subplots_adjust(right = 0.6, top = 1)
90 plot.show()
91
92 w = np.array([np.dot(proj_data, img) for img in normalize_training_tensor])
93 print(w.shape)
94
95 #Function to calculate the best accuracy and the best t0 value
96 def evaluate_performance(t0, testing_tensor, training_tensor, mean_face,
97     ↪ proj_data, w):
98     correct = 0
99     total = len(testing_tensor)
100     for imgNum in range(total):
101         unknown_face_vector = testing_tensor[imgNum,:]
102         normalize_uface_vector = np.subtract(unknown_face_vector, mean_face)
103         w_unknown = np.dot(proj_data, normalize_uface_vector)
104         diff = w - w_unknown
105         norms = np.linalg.norm(diff, axis = 1)
106         idx = np.argmin(norms)
107         if norms[idx] < t0:
108             setNum = int(imgNum / 3)
109             if (idx >= (7*setNum) and idx < (7*(setNum + 1))):
110                 correct += 1
111         else:
112             if (imgNum >= 43):
113                 correct += 1
114         accuracy = correct / total
115     return accuracy
116
117 #Find the best t0 value for the best accuracy
118 t0_values = [1000000*i for i in range(20, 80)]
119 best_t0 = None
120 best_accuracy = 0
121 for t0 in t0_values:
```

```

122     accuracy = evaluate_performance(t0, testing_tensor, training_tensor,
    ↪     mean_face, proj_data, w)
123     if accuracy > best_accuracy:
124         best_accuracy = accuracy
125         best_t0 = t0
126
127     print(f"The most optimal t0: {best_t0} with the best accuracy:
    ↪     {round(best_accuracy, 4)}")
128
129     #Recognition the face with the best t0 and the training tensor
130     count = 0
131     num_img = 0
132     correct = 0
133     def recognition(imgNum, proj_data, w, best_t0):
134         global count, highest_min, num_img, correct
135         num_img += 1
136         #Add the unknown image to the plot
137         unknown_face_vector = testing_tensor[imgNum, :]
138         normalize_uface_vector = np.subtract(unknown_face_vector, mean_face)
139         plot.subplot(14, 6, 1 + count)
140         plot.imshow(unknown_face_vector.reshape(img_height, img_width), cmap =
    ↪         'gray')
141         plot.title("Input: " + str(imgNum + 1), fontsize = 10)
142         plot.tick_params(labelleft = False, labelbottom = False, bottom = False,
    ↪         top = False, left = False, right = False, which='both')
143         count += 1
144
145         w_unknown = np.dot(proj_data, normalize_uface_vector)
146         diff = w - w_unknown
147         norms = np.linalg.norm(diff, axis = 1)
148         idx = np.argmin(norms)
149         plot.subplot(14, 6, count + 1)
150         setNum = int(imgNum/3)
151         t0 = best_t0
152
153         #Find the matched images
154         if norms[idx] < t0:
155             if (idx >= (7*setNum) and idx < (7*(setNum + 1))):
156                 plot.title("Matched with: " + str(idx + 1), color = 'g', fontsize
    ↪                 = 10)

```

```
157         plot.imshow(training_tensor[idx, :].reshape(img_height,
158             ↪ img_width), cmap = 'gray')
159     correct += 1
160     else:
161         plot.title("Matched with: " + str(idx + 1), color = 'r', fontsize
162             ↪ = 10)
163         plot.imshow(training_tensor[idx, :].reshape(img_height,
164             ↪ img_width), cmap = 'gray')
165     else:
166         if (imgNum >= 43):
167             plot.title("Unknown face!", color = 'g', fontsize = 10)
168             correct += 1
169         else:
170             plot.title("Unknown face!", color = 'r', fontsize = 10)
171     plot.tick_params(labelleft = False, labelbottom = False, bottom = False,
172         ↪ top = False, left = False, right = False, which = 'both')
173     plot.subplots_adjust(right = 0.6, top = 1)
174     count += 1
175
176 fig = plot.figure(figsize = (40, 40))
177 for i in range(len(testing_tensor)):
178     recognition(i, proj_data, w, best_t0)
179
180 plot.show()
181 print("Accuracy: {}/{} = {}".format(correct, num_img, round(correct/num_img,
182     ↪ 4)))
```

4.4 Giải thích các câu lệnh

Tên hàm	Ý nghĩa
<code>ndarray</code>	Khởi tạo mảng N chiều
<code>imread</code>	Đọc hình ảnh từ đường dẫn file
<code>flatten</code>	Làm phẳng hàm để trả về bản sao của mảng dạng 1 chiều
<code>subplot</code>	Lấy ba đối số mô tả bố cục của đồ thị
<code>zeros</code>	Tạo mảng các phần tử 0 với chiều và kích thước mong muốn
<code>reshape</code>	Trả về hình dạng mới của mảng mà không làm thay đổi dữ liệu
<code>dot</code>	Thực hiện phép nhân ma trận hoặc tích vô hướng
<code>eig</code>	Tính trị riêng (eigenvalues) và vector riêng (eigenvectors)
<code>sort</code>	Trả về danh sách đã được sắp xếp theo thứ tự
<code>cumsum</code>	Tính tổng tích lũy của các phần tử mảng trên một trục nhất định
<code>transpose</code>	Chuyển vị ma trận
<code>norm</code>	Tính norm của vector hoặc ma trận
<code>argmin</code>	Trả về chỉ số của phần tử nhỏ nhất của mảng trên một trục cụ thể
<code>round</code>	Trả về một số thập phân đã được làm tròn
<code>print</code>	Hiển thị kết quả ra màn hình khi thực thi chương trình
<code>subplot_adjust</code>	Dùng để điều chỉnh bố cục của đồ thị con (subplot)
<code>tick_params</code>	Thay đổi giao diện của đường lưới
<code>show</code>	Hiển thị tất cả các hình
<code>len</code>	Trả về chiều dài của một đối tượng
<code>figure</code>	Tạo hình mới với kích cỡ tùy ý

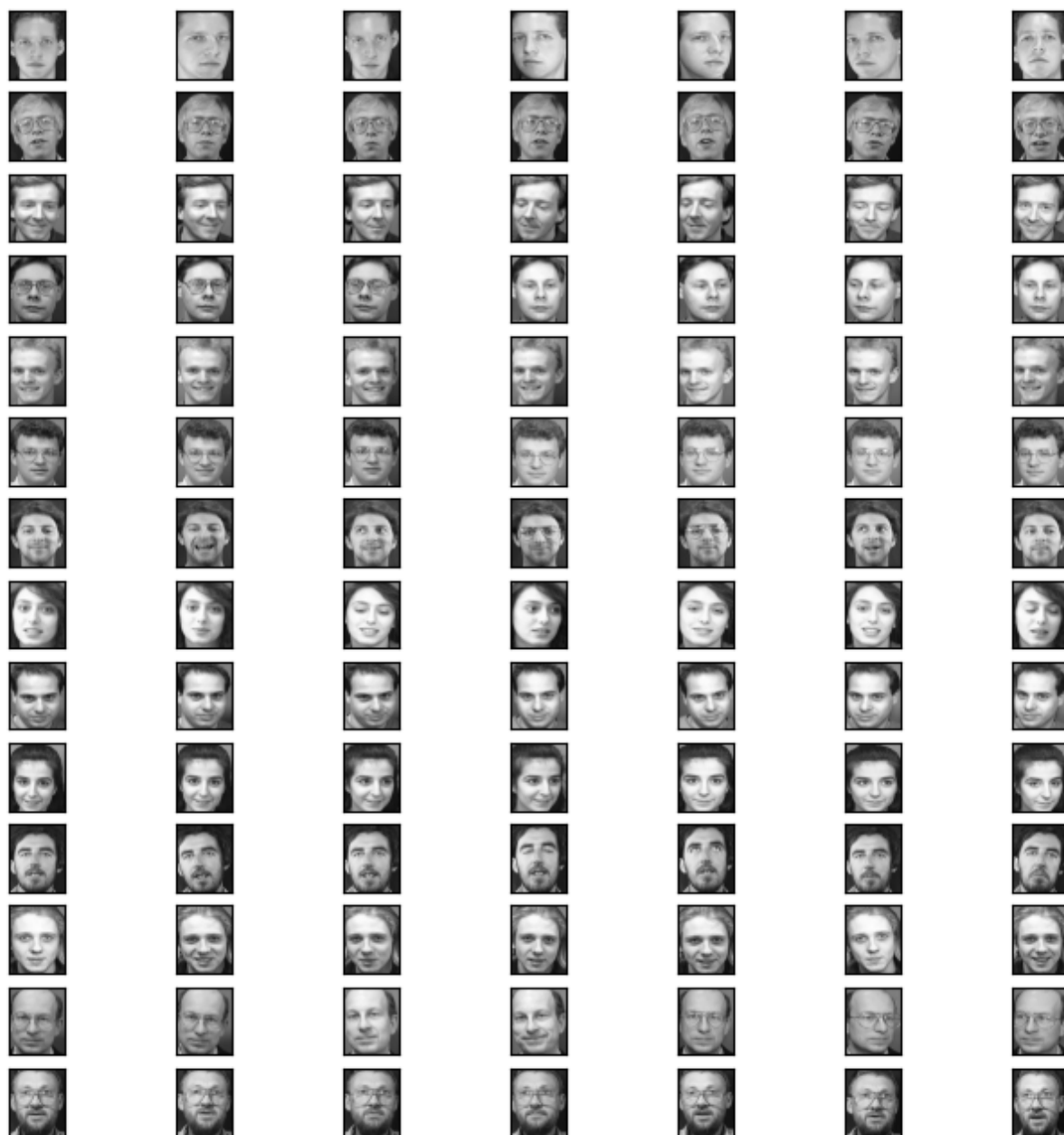
4.5 Kiểm tra và phân tích kết quả

Từ nguồn dữ liệu được đề cập ở Phần 4.3, dữ liệu nhóm sử dụng bao gồm 40 folder tương ứng với 40 người khác nhau được đặt tên từ `s1`, `s2` cho đến `s40` và mỗi folder chứa 10 ảnh khác nhau. Mỗi ảnh được đặt tên từ `1.pgm` đến `10.pgm`.

Nhóm tiến hành lấy 10 ảnh từ folder `s1` đến folder `s14`, trong đó 7 ảnh đầu (từ `1.pgm` đến `7.pgm`) được đưa vào tập training và 3 ảnh cuối (từ `8.pgm` đến `10.pgm`) được đưa vào tập testing.

Tổng quát hơn, dữ liệu của nhóm gồm có 2 tập, tập training bao gồm 98 ảnh (7 ảnh của 14 người khác nhau), tập testing bao gồm 42 ảnh (3 ảnh của 14 người khác nhau).

Hình ảnh trực quan dữ liệu của tập training và tập testing được thể hiện lần lượt qua Hình 4 và Hình 5.

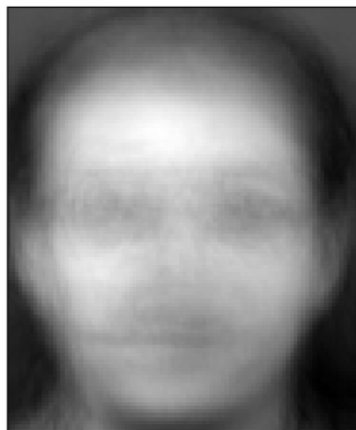


Hình 4: Dữ liệu tập training



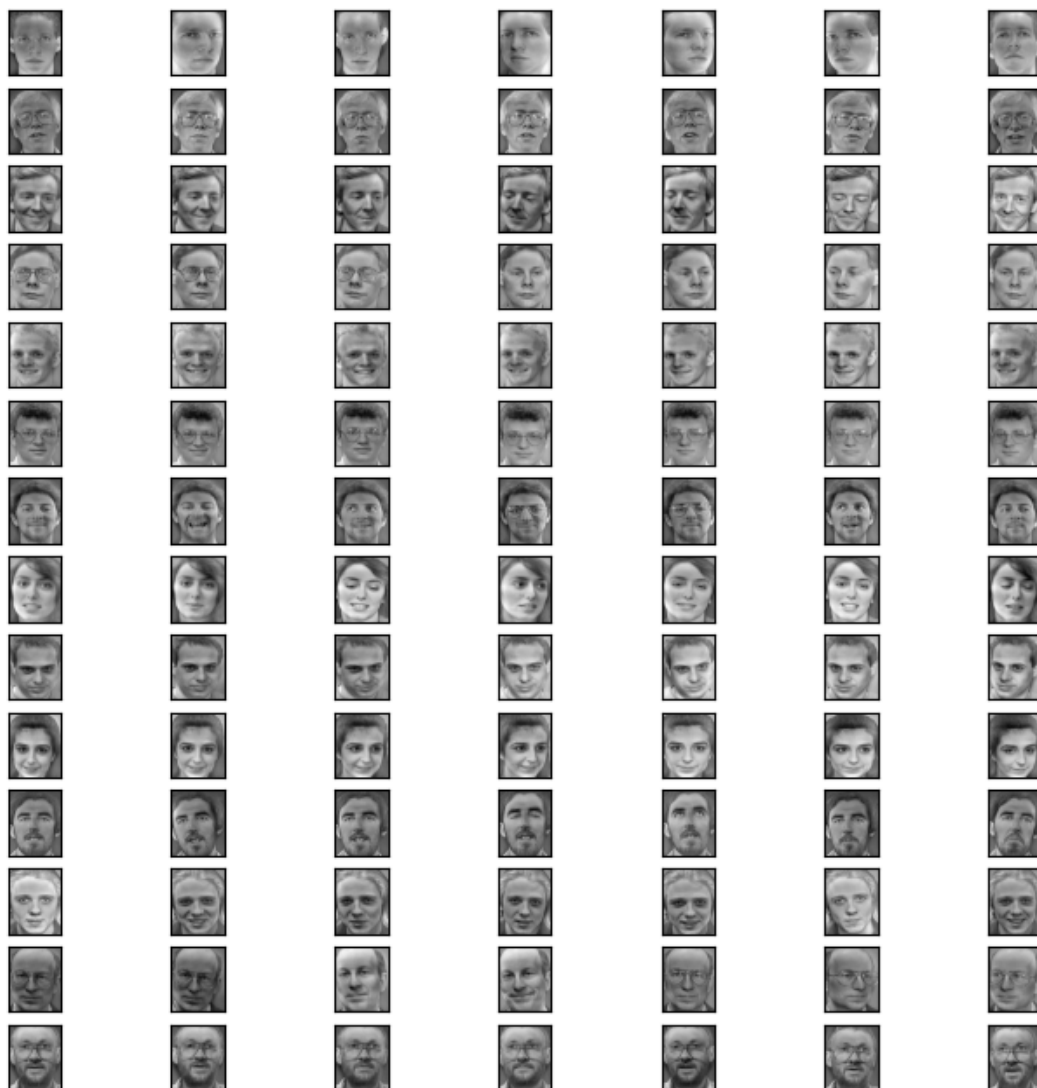
Hình 5: Dữ liệu tập testing

Với dữ liệu từ tập training, ta tiến hành tính trung bình của từng ảnh (mean face) và được thể hiện qua Hình 6.



Hình 6: Trung bình của từng ảnh (mean face)

Sau đó, ta lần lượt trừ đi giá trị trung bình vừa tìm được cho từng ảnh nhằm chuẩn hoá các ảnh. Kết quả thu được thể hiện qua Hình 7.



Hình 7: Chuẩn hoá từng ảnh đầu vào (mean face)

Kế tiếp, ta tiến hành tính ma trận hiệp phương sai, đồng thời tính toán trị riêng và vector riêng của từng ảnh. Sau đó ta tìm kiếm các **eigenfaces** tương ứng với trị riêng, lọc lại những **eigenfaces** lớn nhất và thu được kết quả như Hình 8.



Hình 8: Eigenfaces

Kết quả nhận diện của nhóm được thể hiện qua Hình 9. Trong đó, dòng chữ màu xanh lá (**Matched with**) tương ứng với việc đã nhận diện thành công, còn dòng chữ màu đỏ tương ứng với việc nhận diện sai (**Matched with**) hoặc không nhận diện được (**Unknown face!**).



Hình 9: Kết quả nhận diện khuôn mặt

Từ kết quả thu được ở trên, ta có thể tính được độ chính xác (accuracy) của chương trình bằng cách lấy số ảnh được dự đoán đúng chia tổng số ảnh cần dự đoán.

$$\text{Accuracy} = \frac{40}{42} \approx 0.9524 \quad (16)$$

Ta nhận thấy, với ảnh cần nhận diện là ảnh **3.pgm** thì kết quả cho ra ảnh không trùng khớp (ảnh **90.pgm**), còn đối với ảnh **30.pgm** thì kết quả lại không thể nhận diện được khuôn mặt. Nguyên nhân xảy ra có thể giá trị ngưỡng khoảng cách Euclidean nhỏ hơn so với khoảng cách Euclidean của hai cặp ảnh đó.

Do đó, ta có thể tăng ngưỡng khoảng cách lên một giá trị nào đó để có thể nhận diện được chính xác hai khuôn mặt đó, tuy nhiên nếu ngưỡng khoảng cách quá cao có thể dẫn đến tình trạng chương trình thông báo nhận diện chính xác nhưng ảnh lại khác nhau.

Tương tự, khi ngưỡng khoảng cách quá nhỏ, hệ thống sẽ chỉ nhận diện những khuôn mặt rất giống với khuôn mặt trong tập training. Điều này có thể dẫn đến nhiều trường hợp khuôn mặt đã biết bị coi là không biết.

Bên cạnh đó, việc nhận diện sai hoặc không nhận diện được cũng có thể do việc giữ lại số lượng thành phần chính (các **eigenfaces**).

Nếu như giữ lại quá ít thành phần chính, ta có thể mất đi những thông tin quan trọng cần thiết cho việc nhận diện chính xác. Hoặc nếu giữ lại quá nhiều thành phần chính, ta có thể dẫn đến việc mô hình học cả những chi tiết không quan trọng hoặc nhiễu trong dữ liệu.

Ta có thể khái quát số lượng thành phần chính được giữ lại thông qua biểu đồ phương sai tích lũy, trong đó trục x biểu diễn số lượng thành phần chính được giữ lại, trục y biểu diễn phần trăm phương sai tổng được giữ lại khi thêm từng thành phần chính một.

Thông thường, người ta chọn số lượng thành phần chính mà tại đó tỷ lệ phương sai tích lũy đạt ít nhất 90% hoặc 95%. Do đó, từ biểu đồ trên, số lượng thành phần chính nên giữ lại nằm trong khoảng từ 50 đến 80.

Điều này giúp loại bỏ nhiễu trong mô hình, đồng thời tăng hiệu suất tính toán của chương trình mà vẫn giữ lại đủ thông tin.

Tổng kết

Qua quá trình nghiên cứu và phân tích, chúng ta thấy được PCA đóng vai trò to lớn và có nhiều ứng dụng trong lĩnh vực nhận diện khuôn mặt cũng như các lĩnh vực liên quan. PCA giúp chúng ta hiểu rõ hơn về cách hoạt động của nó và nhấn mạnh tầm quan trọng của việc giảm số chiều dữ liệu và trích xuất những thành phần quan trọng nhất. Nhờ đó, chúng ta có thể xây dựng các mô hình nhận diện có độ chính xác cao.

Không chỉ hữu ích trong việc xây dựng mô hình nhận diện khuôn mặt, PCA còn được ứng dụng rộng rãi trong các hệ thống xác thực và kiểm soát truy cập trong lĩnh vực an ninh. Bên cạnh đó, PCA cũng được tích hợp trong các ứng dụng và thiết bị di động, cho phép người dùng nhận diện khuôn mặt một cách tiện lợi và nhanh chóng.

Tuy nhiên, PCA vẫn đối mặt với một số thách thức cần phải vượt qua. Cụ thể, PCA không thể xử lý tốt các biến đổi trên khuôn mặt do tác động từ môi trường hoặc các yếu tố ngoại cảnh, và gặp khó khăn trong việc xử lý các dữ liệu không ổn định. Điều này đòi hỏi chúng ta phải tiếp tục nghiên cứu và cải tiến hệ thống nhận diện khuôn mặt để đảm bảo tính ổn định và độ chính xác cao.

Ngoài ra, trong tương lai, PCA còn có tiềm năng to lớn trong việc phát triển và mở ra những hướng đi mới trong lĩnh vực nhận diện khuôn mặt. Những nghiên cứu và cải tiến tiếp theo sẽ giúp khai thác tối đa khả năng của PCA, mang lại những tiến bộ vượt bậc trong công nghệ nhận diện khuôn mặt và các ứng dụng liên quan.

Tóm lại, PCA không chỉ là một công cụ mạnh mẽ trong việc trích xuất đặc trưng từ hình ảnh khuôn mặt mà còn là nền tảng cho nhiều ứng dụng công nghệ cao, góp phần nâng cao chất lượng và hiệu quả của các hệ thống nhận diện khuôn mặt trong thực tế.

Tài liệu

- [1] Vũ Hữu Tiếp (15/06/2017). *Bài 27: Principal Component Analysis (phần 1/2)*. Truy cập tại <https://machinelearningcoban.com/2017/06/15/pca/>.
- [2] Vũ Hữu Tiếp (21/06/2017). *Bài 27: Principal Component Analysis (phần 2/2)*. Truy cập tại <https://machinelearningcoban.com/2017/06/21/pca2/>.
- [3] Đặng Văn Vinh (2022), *Đại số tuyến tính*, NXB Đại học Quốc gia TP. Hồ Chí Minh.